

EXAMEN.md — Alcance del proyecto

Este documento es el enunciado del examen: la fuente de verdad sobre **qué hay que construir y entregar**. No describe cómo está construido el sistema — para eso están CLAUDE.md, docs/ y las specs.

Precedencia: si algo de este documento contradice a una decisión del proyecto, gana este documento y la decisión se revisa. Lo que este documento no menciona, lo gobiernan las reglas del proyecto.

Un mínimo no es un techo. Donde el enunciado pide «tres roles como mínimo» o «al menos dos invariantes», tener más no es un incumplimiento.

1 · Configuración

- Un **agente entrevistador** recoge los datos del destinatario: nombre, edad, rasgos, recuerdos, género, tono y extensión. También las palabras o temas que el cliente no quiere que aparezcan.
- Detecta los datos que faltan y **al menos un tipo de contradicción** (por ejemplo, edad frente a género o tono).
- El usuario puede pegar **texto libre** (una anécdota, una carta) del que se extraen hechos. Ese texto se trata como **contenido no confiable**.
- El resultado de la entrevista es un **brief estructurado y validado con schema**.

2 · Lectura interactiva (web o PDF)

La novela se entrega como web o como PDF interactivo. En ambos casos debe incluir:

- Un **índice de capítulos navegable**.
- Una **ficha de personajes y lugares** generada desde la story bible, **con enlaces al capítulo donde aparece cada uno**.
- Una **portada con dedicatoria personalizada**.

Si es web: el lector puede seleccionar un fragmento o un hecho y pedir un cambio desde la propia página («el perro se llama Nala»). El sistema identifica los capítulos que usan ese hecho, **regenera solo esos sin romper la continuidad** y marca en la lectura qué capítulos han cambiado respecto a la versión anterior.

Si es PDF: el cambio se pide desde fuera del documento (formulario o CLI) y se genera una versión nueva del PDF, con una página inicial de «novedades» que lista los capítulos modificados y enlaza internamente a cada uno.

En ambas opciones **se conserva la versión anterior** de la novela.

3 · Harness

- **Tres roles como mínimo:** planner, writer y editor/critic.
- Un archivo de instrucciones CLAUDE.md, **una skill reutilizable** y **dos hooks:** uno de validación del capítulo y otro de policy.
- **Tools con schema validado.**
- **Retries con límite.**
- Registro de **tokens y coste por novela a través de Langfuse** (ver §6).

4 · Memoria

- Una **story bible en SQLite (obligatorio)** en la que **cada hecho registra en qué capítulos se usa**. Incluye una **tabla de cronología** (eventos, momento, personajes, lugar) que alimenta el validador formal.
- **Resúmenes por capítulo** para construir el contexto de los siguientes.
- **Checkpoint por capítulo**: si la generación falla, se reanuda desde el último capítulo completado.

5 · Validación y evaluación

El sistema debe incluir validadores de cuatro tipos. Cada validador **tiene un nombre, se ejecuta en un punto concreto del harness** (hook, rol editor o gate antes de publicar una versión) y **envía su resultado a Langfuse como score**.

a) Validadores programáticos (deterministas) — mínimo tres

- El brief y la salida de cada rol cumplen su schema.
- El nombre del destinatario y los personajes aparecen **escritos exactamente** como en la story bible.
- La **longitud de cada capítulo** está dentro del rango.
- **Cada elemento personalizado obligatorio** del brief aparece en al menos un capítulo, comprobado contra la tabla de hechos de SQLite.
- El **guardrail de palabras prohibidas** (ver §7).
- **Validación visual vía browser MCP**: el agente abre la novela en el navegador, navega por los capítulos y verifica que el índice, la ficha de personajes y la portada renderizan correctamente. Si detecta un error visual, lo registra como fallo y lo devuelve al writer o al rol correspondiente.

b) Validadores no programáticos (semánticos) — mínimo dos

- Un **LLM-as-judge con rúbrica** que evalúe continuidad, tono, calidad narrativa (arco de la historia, coherencia de personajes, ritmo entre capítulos) y que la **personalización esté integrada de forma natural y no forzada**, con **una puntuación por criterio y una justificación**.
- Una **revisión humana** de al menos una novela completa, con la misma rúbrica, para comparar el juicio humano con el del LLM.

c) Validador formal de la historia — Lean 4

- A partir de la story bible en SQLite se genera un fichero Lean con los hechos temporales: **eventos, momento, personajes presentes, lugar, fechas de nacimiento**.
- **Al menos dos invariantes**, por ejemplo:
 - los eventos respetan el orden temporal declarado;
 - la edad de un personaje en cada evento es coherente con su fecha de nacimiento;
 - un personaje no está en dos lugares en el mismo momento;
 - un personaje no aparece después de un evento que lo excluye (muerte, partida definitiva).
- La verificación se ejecuta **de forma automática** (lake build o lean) y, **si falla, la versión de la novela no se publica** y el fallo vuelve al editor como feedback.
- **Debe mostrarse al menos un caso real** en el que el validador formal detecta una incoherencia que los otros validadores no detectaron, o justificar por qué no se encontró

ninguno.

d) Validador formal del sistema — TLA+

Mientras Lean verifica la coherencia de la historia, **TLA+ verifica el comportamiento del harness**. Referencia: learntla.com.

- Una especificación en TLA+ o PlusCal del flujo de generación como máquina de estados: **configuración → planificación → escritura de capítulo → validación → publicación de versión**, incluyendo **retries, reanudación desde checkpoint y regeneración por cambio del lector**.
- **Al menos tres invariantes de seguridad**, por ejemplo:
 - nunca se publica una versión con un capítulo que no ha pasado todos los validadores;
 - la reanudación desde checkpoint no duplica ni pierde capítulos;
 - la versión anterior de la novela se conserva siempre tras una regeneración;
 - el número de reintentos nunca supera el límite.
- **Al menos una propiedad de liveness**: toda generación termina publicando una versión o deteniéndose con error; nunca queda en un bucle infinito.
- Verificación con el model checker **TLC** sobre un modelo pequeño (por ejemplo, 5 capítulos y 2 reintentos), **con la configuración incluida en el repo**.
- **La especificación debe corresponder al código**: el README explica qué estado o transición del código implementa cada acción de la especificación.
- Si TLC encontró algún contraejemplo durante el desarrollo, **se documenta junto con el cambio que hizo en el código**.

Evaluación del sistema

- **Cinco briefs de prueba**, incluido **al menos uno adversarial** (injection en el texto libre) y **uno diseñado para provocar una incoherencia temporal**.
- Una **tabla que muestre, por brief, qué validadores pasaron y cuáles fallaron**.
- Una **iteración de tuning documentada**, con los resultados antes y después.

6 · Observabilidad

- Cada generación de novela es **una traza en Langfuse, agrupada por sesión** — una sesión por novela, incluyendo la entrevista y las regeneraciones posteriores.
- **Cada rol** (entrevistador, planner, writer, editor) y **cada llamada a tool** aparece como **span con nombre identificable**.
- **Tokens, coste y latencia** visibles **por llamada, por capítulo y por novela**.
- Los resultados de **todos** los validadores (programáticos, semánticos y Lean) se envían a Langfuse **como scores** asociados a su traza. TLC se ejecuta en desarrollo, no en cada generación.
- **Los prompts versionados en Langfuse**, de forma que la iteración de tuning muestre qué versión de prompt produjo cada resultado.

7 · Guardrails

Un **guardrail de palabras prohibidas**, aplicado **en código** sobre cada capítulo **antes de aceptarlo**:

- Listas guardadas en SQLite, **en tres niveles**: globales (insultos, términos ofensivos) y por novela, definidas por el cliente en la configuración (por ejemplo, el nombre de una expareja o un tema que no quiere que aparezca).

- La detección **normaliza el texto antes de comparar**: mayúsculas, acentos, plurales y variantes simples.
- Si hay coincidencia, el capítulo **se devuelve al writer** para reescribirlo, **con un límite de intentos**. Si se agota el límite, **la generación se detiene y se informa**.
- **Cada coincidencia queda registrada en el audit log y en Langfuse**.
- Tests que cubran **al menos un caso de cada nivel** y **un caso de variante** (acento o plural).

Más un **audit log de las decisiones del policy engine**.

Restricción global

Uso de un máximo de **100.000 tokens concurrentes**.

Fuera de alcance

Pagos, cuentas de usuario, impresión física, ilustraciones, audio y despliegue en producción.

Los repositorios deben incluir también

Novela de ejemplo generada

El **PDF de una novela completa de 10 capítulos**, generada con el brief de ejemplo del README, commiteada en /ejemplos/novela-ejemplo.pdf. Es la evidencia de que el sistema funciona de principio a fin. **Si el formato de lectura elegido es web, se incluye igualmente el PDF exportado**.

/docs en storyMaker, con la documentación de proceso

No se corrige el resultado, se corrige el razonamiento que llevó a él.

- **Spec inicial**: qué se decidió construir y por qué, antes de escribir código.
- **Trade-offs**: cada decisión de diseño relevante explicada como decisión — opciones, criterios y elección. Por ejemplo: single-agent frente a multi-agent, formato de la story bible, elección del modelo de lectura, integración de TLA+ con el flujo real, invariantes de Lean priorizados.
- **Explainers**: uno por cada concepto del curso aplicado en el proyecto. Breves. Para demostrar que se entiende lo que se aplica, no para copiar la teoría.
- **Diagramas**: arquitectura del harness, máquina de estados de TLA+, esquema SQLite, tabla de validadores con su punto de ejecución.
- **Registro de iteraciones**: qué cambió tras cada eval o contraejemplo de TLC o Lean, y por qué. No un diario, sino un log de decisiones con causa y efecto.
- **Red-team log**: casos adversariales probados, qué validador los detectó (o no) y cómo se resolvió.

Vídeo de demo

En cualquier formato (Loom, MP4 u otro), de la duración que se considere necesaria para mostrar el sistema con claridad. Debe estar subido al repositorio storyMaker dentro de /presentacion/, o enlazado desde su README.md si el fichero supera el límite de tamaño de

GitHub.

Sin API keys en ningún repo

Usar `.env.example`.

Claude Code

Los estudiantes trabajan con Claude Code. El repo debe reflejar ese uso:

- El fichero **CLAUDE.md en la raíz** del repo —ya obligatorio como archivo de instrucciones del harness— **debe estar cuidado y ser legible: es parte del examen**.
- La carpeta **.claude/** con los ficheros de memoria y comandos personalizados **debe estar commiteada**.
- El fichero de configuración MCP (`.claude/mcp.json` o equivalente) debe incluir **un servidor MCP de inspección de browser** (Chrome MCP, Playwright MCP o similar), de forma que Claude Code pueda abrir la lectura web de la novela y verificar el resultado visualmente.
- **El uso real del browser MCP debe estar documentado en /docs:** qué inspeccionó el agente, qué detectó y qué cambio provocó en el código o en los prompts.
- Los ficheros de **skills** usados o creados durante el desarrollo deben estar en el repo y **referenciados desde /docs**.
- Si se han usado **subagentes o comandos / propios**, deben estar documentados en /docs con su propósito y resultado.

Un proyecto sin evals con resultados medibles, o sin documentación de proceso en /docs, no aprueba.